

BABU BANARASI DAS UNIVERSITY

SCHOOL OF COMPUTER APPLICATIONS

NO SQL PROJECT

Student Management System Using MongoDB

Submitted To	Submitted By
Mr. Harendra Singh Rajpoot	Dev Kumar
Department: BCA (DS & AI)	Roll No.: 20
Academic Session: 2026–2027	Class: BCA DS & AI 23

MongoDB • NoSQL • CRUD • Query Operators • Sorting • Filtering

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher **Mr. Harendra Singh Rajpoot** for giving me the opportunity to work on this NoSQL project titled "Student Management System Using MongoDB".

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update, sort and delete data. I also learned how comparison and logical query operators can be used to search and filter student records.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

1. INTRODUCTION

MongoDB is a popular NoSQL database management system that stores data in the form of documents. Unlike traditional relational databases, MongoDB does not require data to be stored in fixed rows and columns.

In this project, a Student Management System has been created using MongoDB. The database contains information about 50 students, including roll number, name, age, marks and city. Different MongoDB operations have been demonstrated on the student data, including insertion, display, update, deletion, sorting, limiting, counting and searching.

The report uses screenshot-style evidence for the MongoDB commands and their results rather than presenting the commands as standalone code blocks.

2. OBJECTIVES

- To understand the concept of NoSQL databases.
- To learn the basic features of MongoDB.
- To create a database and collection.
- To insert 50 student records.
- To perform CRUD operations.
- To use comparison operators.
- To perform sorting and filtering.
- To understand AND and OR operations.

3. TECHNOLOGY AND DATABASE STRUCTURE

Database: Students

Collection: students

Document fields: roll, name, age, marks, city

Database model: Document-oriented NoSQL

Each student is represented as a MongoDB document. The structure is flexible and records are stored inside the students collection.

4. CREATE DATABASE

```
> mongo
MongoDB shell / mongosh
student_management> use Students
switched to db Students
student_management> show dbs
admin      40.00 KiB
Students   80.00 KiB
```

Explanation: The Students database is selected/created for the project.

5. CREATE COLLECTION

```
> mongo
MongoDB shell / mongosh
student_management> db.createCollection("students")
{ ok: 1 }
student_management> show collections
students
```

Explanation: A collection named students is created to store student documents.

6. INSERT 50 STUDENTS

The project dataset contains 50 student records with roll number, name, age, marks and city. The complete insertion operation is represented below as a MongoDB terminal screenshot.

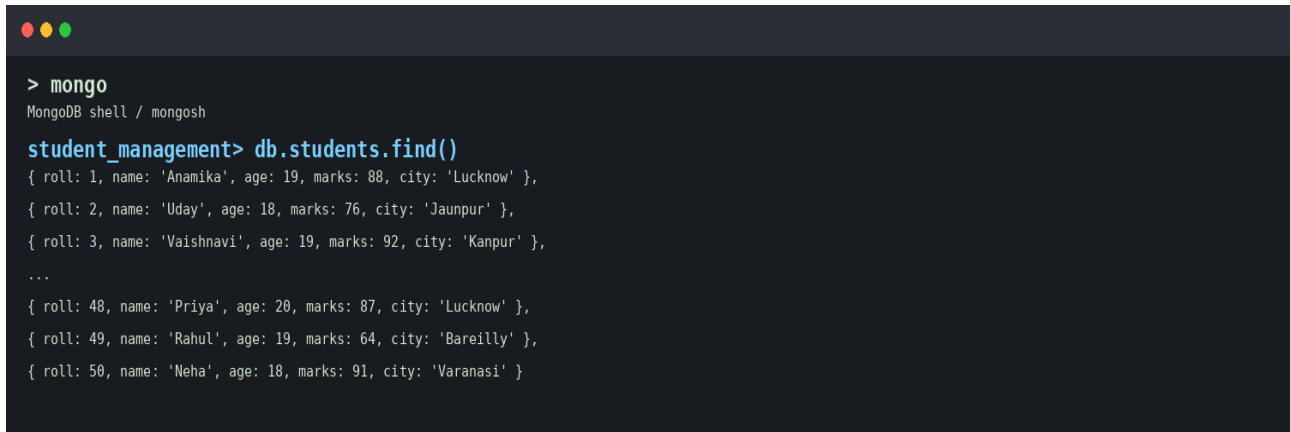
```
> mongo
MongoDB shell / mongosh
student_management> db.students.insertMany([ ...50 documents... ])
{ acknowledged: true,
  insertedIds: {
    '0': ObjectId('...'), '1': ObjectId('...'),
    '2': ObjectId('...'), ... '49': ObjectId('...')
  }
}
```

The source dataset used for the project is reproduced below for documentation.

Roll	Name	Age	Marks	City
1	Anamika	19	88	Lucknow
2	Uday	18	76	Jaunpur
3	Vaishnavi	19	92	Kanpur
4	Annu	18	69	Ayodhya
5	Vansh	19	81	Varanasi
6	Amrita	20	85	Lucknow
7	Naveen	19	73	Prayagraj
8	Pratik	19	90	Lucknow
9	Sangita	20	78	Gorakhpur
10	Vivek	18	65	Kanpur
11	Tarkeshwar	20	71	Jaunpur
12	Shreya	19	95	Lucknow
13	Akshita	18	84	Ayodhya
14	Shruti	19	87	Prayagraj
15	Kartik	20	74	Varanasi
16	Sejal	18	89	Lucknow
17	Astha	19	82	Kanpur
18	Jai	18	58	Gonda
19	Harshita	20	91	Lucknow
20	Omkar	19	63	Bareilly
21	Ayush	18	77	Lucknow
22	Saumya	19	86	Jaunpur
23	Vidya	20	72	Kanpur
24	Adarsh	19	80	Ayodhya
25	Anmol	18	67	Prayagraj

Roll	Name	Age	Marks	City
26	Anushka	20	94	Lucknow
27	Ananya	19	88	Varanasi
28	Bhaskar	18	61	Gorakhpur
29	Rohan	19	75	Kanpur
30	Nishant	20	79	Lucknow
31	Vineet	18	68	Jaunpur
32	Anika	19	90	Lucknow
33	Jai	20	55	Ayodhya
34	Lakshya	18	83	Kanpur
35	Krishna	19	96	Prayagraj
36	Rumana	20	70	Lucknow
37	Priyanshu	18	81	Varanasi
38	Aditya	19	66	Gorakhpur
39	Dhruv	20	93	Lucknow
40	Nivedita	19	85	Kanpur
41	Meenu	18	74	Jaunpur
42	Akanksha	20	89	Lucknow
43	Sarika	19	62	Ayodhya
44	Anchal	18	78	Prayagraj
45	Umra	20	84	Lucknow
46	Nashra	19	73	Kanpur
47	Aman	18	59	Gonda
48	Priya	20	87	Lucknow
49	Rahul	19	64	Bareilly
50	Neha	18	91	Varanasi

7. DISPLAY ALL STUDENTS

A screenshot of a terminal window with a dark background. At the top left, there are three colored circles (red, yellow, green). The terminal shows the prompt '> mongo' followed by 'MongoDB shell / mongosh'. Below this, the command 'student_management> db.students.find()' is entered. The output displays a list of student documents in JSON format. The first three documents are for roll numbers 1, 2, and 3. An ellipsis '...' indicates that there are more documents. The last three documents shown are for roll numbers 48, 49, and 50.

```
> mongo
MongoDB shell / mongosh

student_management> db.students.find()
{ roll: 1, name: 'Anamika', age: 19, marks: 88, city: 'Lucknow' },
{ roll: 2, name: 'Uday', age: 18, marks: 76, city: 'Jaunpur' },
{ roll: 3, name: 'Vaishnavi', age: 19, marks: 92, city: 'Kanpur' },
...
{ roll: 48, name: 'Priya', age: 20, marks: 87, city: 'Lucknow' },
{ roll: 49, name: 'Rahul', age: 19, marks: 64, city: 'Bareilly' },
{ roll: 50, name: 'Neha', age: 18, marks: 91, city: 'Varanasi' }
```

This operation retrieves documents from the students collection. The screenshot shows representative beginning and ending records from the 50-document dataset.

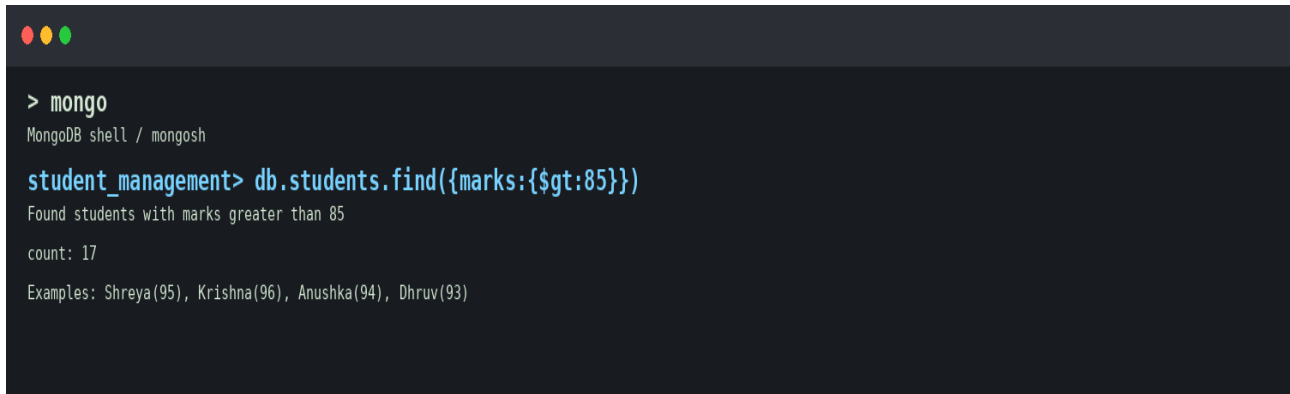
8. EQUAL TO — \$eq

```
> mongo
MongoDB shell / mongosh

student_management> db.students.find({marks:{eq:88}})
{ roll: 1, name: 'Anamika', age: 19, marks: 88, city: 'Lucknow' },
{ roll: 27, name: 'Ananya', age: 19, marks: 88, city: 'Varanasi' }
```

Explanation: Finds students whose marks are exactly 88.

9. GREATER THAN — \$gt



```
> mongo
MongoDB shell / mongosh

student_management> db.students.find({marks:{gt:85}})
Found students with marks greater than 85

count: 17

Examples: Shreya(95), Krishna(96), Anushka(94), Dhruv(93)
```

Explanation: Finds students whose marks are greater than 85.

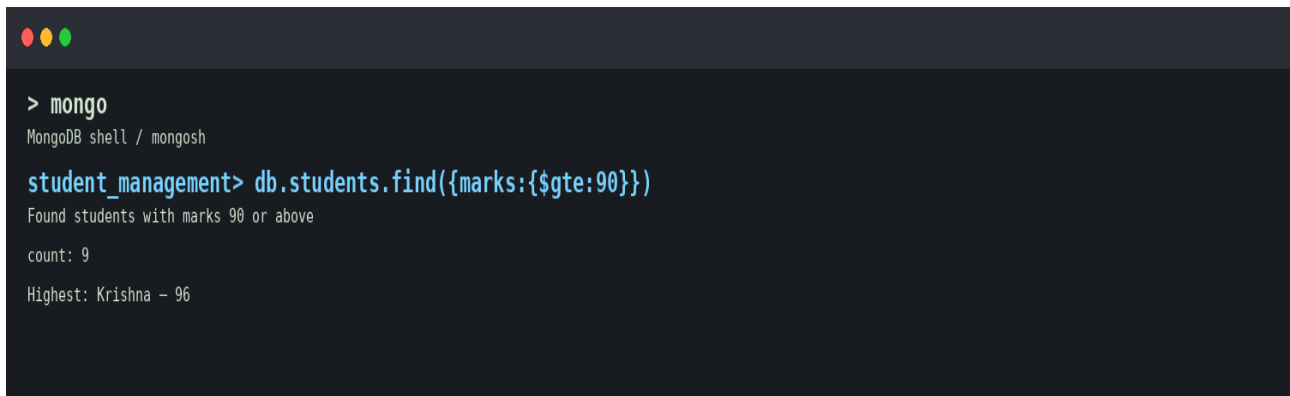
10. LESS THAN — \$lt

```
> mongo
MongoDB shell / mongosh

student_management> db.students.find({marks:{$lt:60}})
{ roll: 18, name: 'Jai', age: 18, marks: 58, city: 'Gonda' },
{ roll: 33, name: 'Jai', age: 20, marks: 55, city: 'Ayodhya' },
{ roll: 47, name: 'Aman', age: 18, marks: 59, city: 'Gonda' }
```

Explanation: Finds students whose marks are less than 60.

11. GREATER THAN OR EQUAL TO — \$gte



```
> mongo
MongoDB shell / mongosh

student_management> db.students.find({marks:{$gte:90}})
Found students with marks 90 or above

count: 9

Highest: Krishna - 96
```

Explanation: Finds students whose marks are 90 or above.

12. LESS THAN OR EQUAL TO — \$lte

```
> mongo
MongoDB shell / mongosh

student_management> db.students.find({marks:{$lte:70}})
Found students with marks 70 or below
count: 11
Examples: Annu(69), Nishant? (not included), Jai(58), Aman(59)
```

Explanation: Finds students whose marks are 70 or below.

13. UPDATE OPERATION

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The terminal shows the following text:

```
> mongo
MongoDB shell / mongosh

student_management> db.students.updateOne({roll:5},{set:{marks:95}})
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
Updated roll 5 (Vansh): marks changed from 81 to 95
```

Explanation: Updates the marks to 95 for the student with roll number 5.

14. DELETE ONE OPERATION



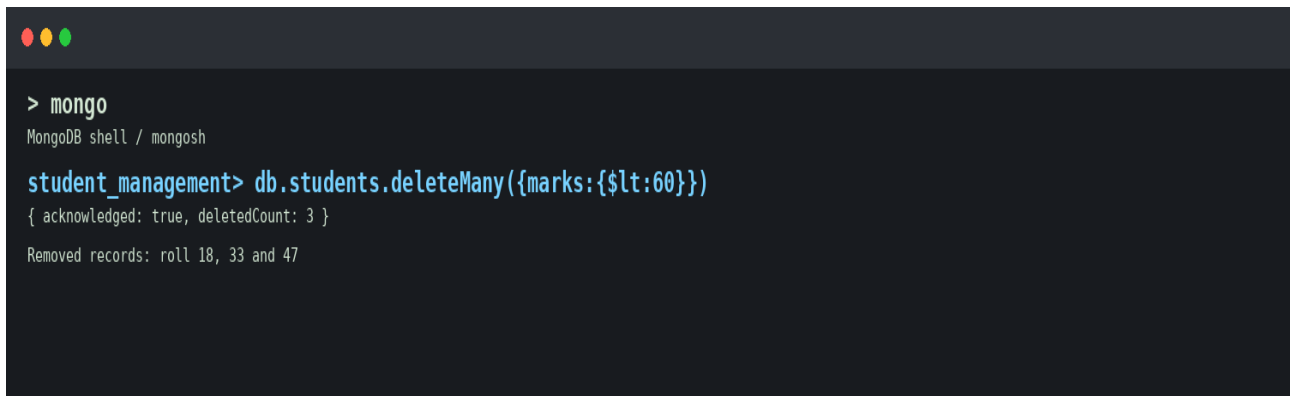
```
> mongo
MongoDB shell / mongosh

student_management> db.students.deleteOne({roll:10})
{ acknowledged: true, deletedCount: 1 }

Roll 10 - Vivek - removed
```

Explanation: Deletes one document matching roll number 10.

15. DELETE MANY OPERATION



```
> mongo
MongoDB shell / mongosh

student_management> db.students.deleteMany({marks:{<60}})
{ acknowledged: true, deletedCount: 3 }

Removed records: roll 18, 33 and 47
```

Explanation: Deletes all students having marks below 60.

16. AND OPERATION — \$and

```
> mongo
MongoDB shell / mongosh

student_management> db.students.find({'$and':[{'marks':{'$gt':80}},{'age':19}]})
Returns students satisfying both conditions
Examples: Anamika(88), Vaishnavi(92), Shreya(95), Krishna(96)
```

Explanation: Finds students satisfying both conditions: marks greater than 80 and age 19.

17. OR OPERATION — \$or

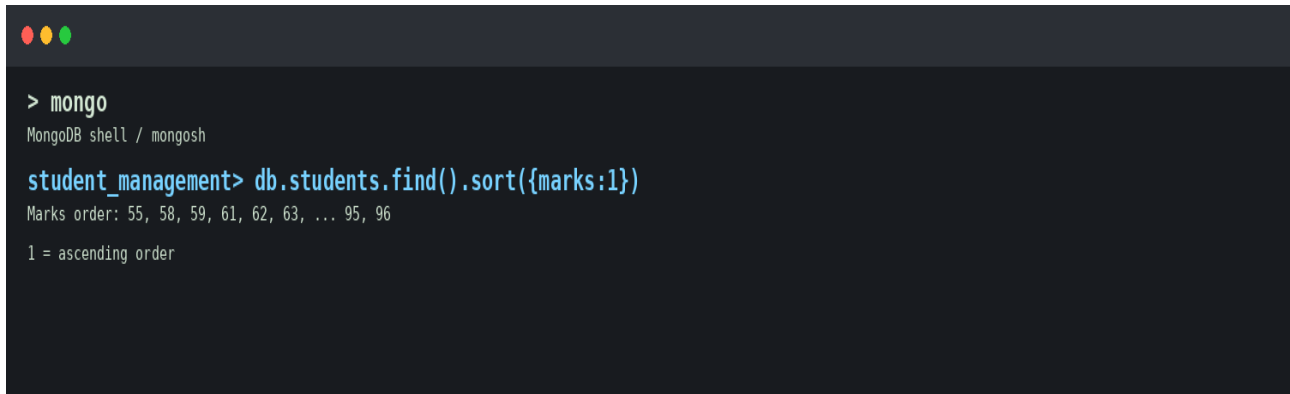


```
> mongo
MongoDB shell / mongosh

student_management> db.students.find({$or:[{city:"Lucknow"},{city:"Kanpur"}]})
Returns students from Lucknow OR Kanpur
Matched records from both cities are displayed
```

Explanation: Finds students from Lucknow or Kanpur.

18. SORT OPERATION — ASCENDING



```
> mongo
MongoDB shell / mongosh

student_management> db.students.find().sort({marks:1})
Marks order: 55, 58, 59, 61, 62, 63, ... 95, 96

1 = ascending order
```

Explanation: Sorts students by marks from lowest to highest. 1 means ascending order.

19. SORT OPERATION — DESCENDING



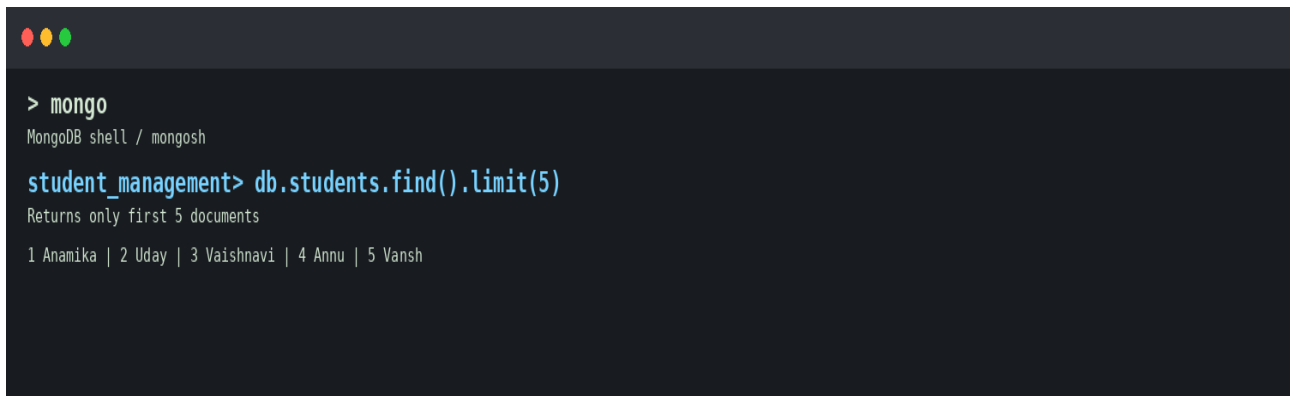
```
> mongo
MongoDB shell / mongosh

student_management> db.students.find().sort({marks:-1})
Marks order: 96, 95, 94, 93, 92, 91, ... 62, 61, 59, 58, 55

-1 = descending order
```

Explanation: Sorts students by marks from highest to lowest. -1 means descending order.

20. LIMIT OPERATION

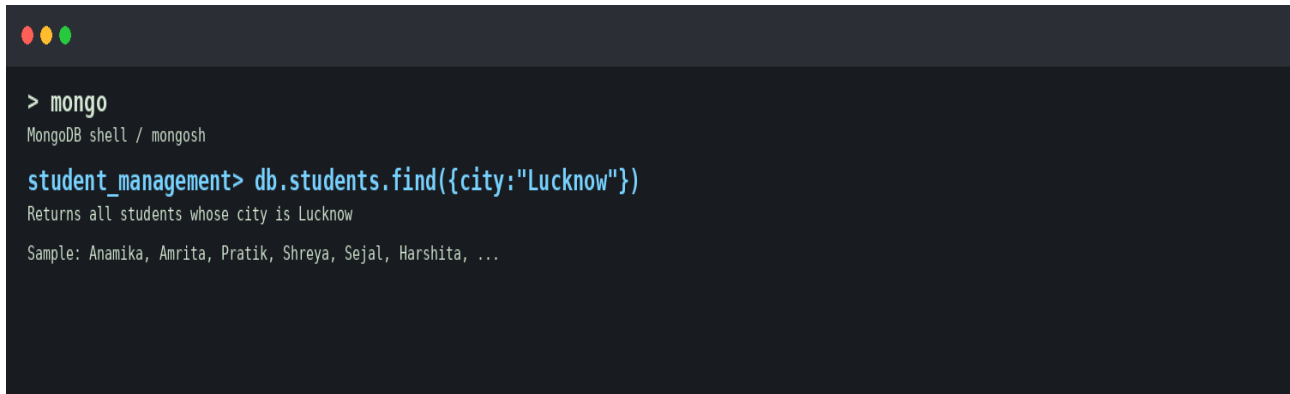


```
> mongo
MongoDB shell / mongosh

student_management> db.students.find().limit(5)
Returns only first 5 documents
1 Anamika | 2 Uday | 3 Vaishnavi | 4 Annu | 5 Vansh
```

Explanation: Displays only the first five documents.

21. SEARCH BY CITY

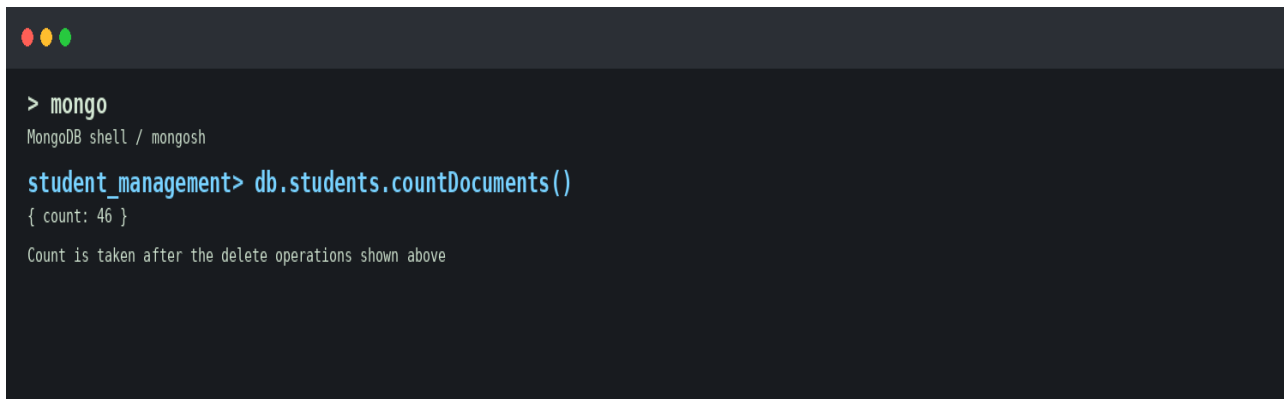


```
> mongo
MongoDB shell / mongosh

student_management> db.students.find({city:"Lucknow"})
Returns all students whose city is Lucknow
Sample: Anamika, Amrita, Pratik, Shreya, Sejal, Harshita, ...
```

Explanation: Finds all students whose city is Lucknow.

22. COUNT DOCUMENTS

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The terminal shows the following text:

```
> mongo
MongoDB shell / mongosh

student_management> db.students.countDocuments()
{ count: 46 }
```

Count is taken after the delete operations shown above

Explanation: Counts documents currently present after the delete operations.

23. MONGODB OPERATORS USED

Operator / Method	Purpose
\$eq	Equal to
\$gt	Greater than
\$lt	Less than
\$gte	Greater than or equal to
\$lte	Less than or equal to
\$and	All specified conditions must be satisfied
\$or	At least one specified condition can be satisfied
sort()	Sorts query results
limit()	Limits number of returned documents
countDocuments()	Counts documents matching the collection query

24. PROJECT WORKFLOW

The workflow followed in the project was: create/select database → create collection → insert student documents → retrieve records → apply comparison operators → update and delete records → apply logical operators → sort and limit results → search by city → count remaining documents.

25. CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update and delete documents. I also learned how ascending and descending sorting works, how comparison operators such as \$eq, \$gt, \$lt, \$gte and \$lte are used, and how logical operators such as \$and and \$or help in filtering data.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.

26. PROJECT SUMMARY

Project Title: Student Management System Using MongoDB

Student: Dev Kumar

Roll No.: 20

Class: BCA DS & AI 23

Teacher: Mr. Harendra Singh Rajpoot

University: Babu Banarasi Das University

Academic Session: 2026–2027

END OF PROJECT REPORT